# Lookking — S8 Final Report

# Lookking — Final Project Report

**Course:** S8 Integrated Project, AI & Big Data, UIR **Author:** Karim Akoudad **Submission window:** May 2026 **Status:** Complete. All technical deliverables shipped.

---

## 0. TL;DR for the prof

| Question | Answer |
|---|---|
| What is Lookking? | A Telegram chatbot that recommends places (gyms, spas, restaurants…) and business leads in Morocco using a 3-agent AI system. |
| What deep learning did you train? | DistilBERT fine-tuned to classify query-candidate match relevance into High / Medium / Low. |
| What real data did you use? | **361 real OSM places** crawled from OpenStreetMap (Nominatim) across 9 cities + 11 categories. |
| What's the model accuracy? | **88.9% on test split**, **80.0% on hand-written unseen holdout set** (honest real-world number). |
| Why multi-agent and not one LLM? | Each agent makes a distinct decision the next one depends on (Intent → Retrieval → Ranking). The 3rd agent is a **hybrid DL+LLM** — numeric match from DistilBERT, natural reason from LLM. |
| Free stack? | Yes — Telegram free, LLM free tier (Gemini / Groq / Cerebras), OpenStreetMap free, all libraries open-source. |

---

## 1. PROBLEM STATEMENT

Local search (“find me X near Y”) is broken in two ways: 1. Google Maps gives a list, but **doesn’t understand intent** — it can’t tell that *“a calm cafe to do some work”* is different from *“a noisy cafe to meet friends”*. 2. Freelancers looking for business leads (“I do video editing for restaurants — who needs me?”) have **no free tool** to find target clients.

Lookking attempts both: place-discovery for users and lead-generation for service providers, in a single conversational interface.

---

## 2. WHAT THE USER SEES (the experience)

Step-by-step demo flow:

1. User opens `@LookkingBot` on Telegram.
2. Sends `/start`.
3. Bot shows two inline buttons: 📍 **Find a Place** | 💼 **Find Leads**.
4. User taps **Find a Place**.
5. Bot: *“Describe what place you want.”*
6. User types: `luxury spa Rabat`.
7. After ~15 s the bot returns:

```
Rank 1 | Royal Spa Rabat | spa | Rabat | Rating: 4.8/5 (synthetic) | Niche: lu
  Match: High Match (96.4%)
  Why: This luxurious spa in Rabat directly matches your wish for a luxury spa
  📍 Map: https://www.google.com/maps?q=33.97...,-6.85...
  🗺️ OSM: https://www.openstreetmap.org/?mlat=33.97...&mlon=-6.85...
Rank 2 | Hammam Palace Rabat | spa | Rabat | ...
Rank 3 | Zen Retreat Rabat   | spa | Rabat | ...
```

8. Two buttons appear: ✅ **Done** | ➕ **Add Info**.
9. If the user clicks **Add Info** and types `but in Casablanca`, the bot **merges** the refinement with the prior query and re-runs the pipeline.

The two Map / OSM links per result mean the user can **click** to verify the place exists in real life — no hallucinations.

---

## 3. SYSTEM ARCHITECTURE


Architecture

Architecture

Top-down breakdown:

```
USER (Telegram)
   │
   ▼
TELEGRAM BOT  ←  state machine: IDLE → MODE_PICK → AWAITING_QUERY → RESULTS → REFINE
   │
   ▼
MULTI-AGENT PIPELINE  (CrewAI, sequential)
   │
   ├─ AGENT 1 — Intent Classifier
   │     in:  raw user text
   │     out: structured JSON {mode, city, category, niche, urgency, constraints}
   │
   ├─ AGENT 2 — Retrieval Strategist          (tool: Search Places / Search Leads)
   │     in:  intent JSON
   │     out: 5 candidate places with name, city, rating, lat/lon, map URLs
   │
   └─ AGENT 3 — Ranking & Explanation         (tool: Score Match → DistilBERT)
         in:  candidate list + original query
         out: top 3 ranked + DL match label + LLM reason + map URLs

DATA / EXTERNAL APIs
   ├─ OpenStreetMap Nominatim API (live)
   ├─ data/places_real.csv (361 OSM places, CSV fallback)
   └─ model/lookking_model.pt (DistilBERT trained weights)
```

Source of truth for the diagram is `scripts/draw_architecture.py`, which produces `docs/architecture.png`.

---

## 4. WHY MULTI-AGENT (the prof's #1 question)

The prof said: *"each agent should have a clear job — not just exist while one does all the work"*. We took this seriously and gave each agent a unique, schema-typed output.

### Agent 1 — Intent Classifier

- **Owns:** language understanding.
- **Outputs:** structured JSON.
- **Why an agent and not a regex?** Casual user text contains synonyms ("traditional", "fancy", "vibe", "real Moroccan food") that a rule-based parser would miss. The LLM normalises this into canonical fields.

### Agent 2 — Retrieval Strategist

- **Owns:** data sourcing.
- **Decisions:** Nominatim live vs CSV fallback; query expansion ("barber" → also search "hairdresser"); deduplication; result count.
- **Why an agent and not a function?** Different intents need different strategies. A pharmacy lookup needs *current opening hours*; a hotel lookup needs *city + niche*. The LLM picks how to shape the tool call.

### Agent 3 — Ranking & Explanation

- **Owns:** scoring + user-facing explanation.
- **Hybrid AI:** the **numeric match score** comes from our fine-tuned **DistilBERT**, the **natural-language reason** comes from the LLM. The DL model can't write English; the LLM can't tell us "this is a 96% match" reliably. Combined, they produce explainable ranked results.

Outputs of one agent are the schema-typed inputs of the next. Removing any one agent breaks the pipeline. This is what makes the system genuinely multi-agent.

---

## 5. THE DEEP LEARNING MODEL

### 5.1 Task

> *Given a free-text user query Q and a candidate place description C, classify the match relevance:* ***Low / Medium / High****.*

This is a textbook **sentence-pair classification** task (same family as MNLI, STS-B). BERT/DistilBERT is the standard tool — no need to invent something custom.

### 5.2 Model

```
[QUERY]: <query>  [SEP]  [CANDIDATE]: <candidate>
                  |
```

```
                ▼
DistilBERT (66M params, transformer 6 layers)
     — Embedding layer:        FROZEN  (no gradient)
     — Transformer layers 0–3: FROZEN  (no gradient)
     — Transformer layers 4–5: TRAINED (fine-tuned)
                │
                ▼ [CLS] vector (768-dim)
Dropout (0.3)
                │
                ▼
MLP head (trained from scratch):
     Linear 768 → 256 + ReLU + Dropout
     Linear 256 → 64  + ReLU + Dropout
     Linear 64  → 3
                │
                ▼ softmax
probabilities over {Low, Medium, High}
```

Total trainable parameters: ~14 M.

### 5.3 Training data — the honest version

We collected **361 real OpenStreetMap places** across **9 Moroccan cities** and **11 categories** (restaurant, cafe, gym, spa, hotel, bakery, bar, hairdresser, pharmacy, fast food, supermarket). Crawl script: `data/collect_real_places.py`.

From those real places we generated **1260 (query, candidate, label) triples** with rule-based labels:

| Label | Rule | Count |
|---|---|---|
| HIGH | candidate's category + city match the query | 722 |
| MEDIUM | category matches, city differs | 249 |
| LOW | category differs entirely (clear mismatch) | 289 |

Builder: `data/build_training_v2.py`. Output: `data/training_data_v2.csv`.

### 5.4 Honest evaluation

To avoid the "100% on synthetic" trap, we built a **30-example holdout** in a different writing style: messy real-user-sounding queries like *"I really need a great place to eat tonight in Rabat"* and *"My back hurts, find me a good spa in Agadir"*. File: `data/holdout_test.csv`.

**Numbers (best epoch, run on 2026-05-17):**

| Metric | Test split (252 unseen pairs from training distribution) | Holdout (30 hand-written unseen-style queries) |
|---|---|---|
| Accuracy | **88.89 %** | **80.00 %** |
| Macro F1 | 0.83 | 0.71 |
| Low-class F1 | 0.83 | 0.74 |
| Medium-class F1 | 0.67 | 0.44 |
| High-class F1 | 0.98 | 0.94 |

The model handles HIGH/LOW well; **MEDIUM is the failure mode** (the model is conservative — under-predicts medium, leans low). This matches expectation: medium = ambiguous label, hardest class.

Training script: `model/train_v2.py`. All numbers saved to `model/metrics_v2.json`. Plots: `model/training_curves.png`, `model/confusion_matrix_v2.png`, `model/confusion_matrix_holdout.png`.

### 5.5 Training hyper-parameters

| Param | Value |
|---|---|
| Optimizer | AdamW |
| Learning rate (BERT body) | $2 \times 10^{-5}$ |
| Learning rate (head) | $1 \times 10^{-3}$ |
| Weight decay | 0.01 |
| Gradient clipping | norm 1.0 |
| Batch size | 16 |
| Max sequence length | 128 tokens |
| Epochs | 8 |
| Train / test split | 80 / 20, stratified |
| Device | Apple M-series MPS |

## 6. LIVE DATA INTEGRATION

We did **not** stop at synthetic data. The bot crawls real places from **OpenStreetMap** at runtime via the **Nominatim** API.

- **Tool:** `tools/nominatim_tool.py` (rate-limited to 1.2 s/call to respect the free public API).
- **Country scope:** `countrycodes=ma` (Morocco only).
- **Output enrichment:** synthetic rating from OSM "importance" score; synthetic price level; Google Maps link + OSM link.
- **Fallback:** if Nominatim is empty/slow, we serve from `data/places_real.csv` (the 361 places we crawled offline).

Every place returned to the user is **verifiable**: tap the Google Maps link, see the pin.

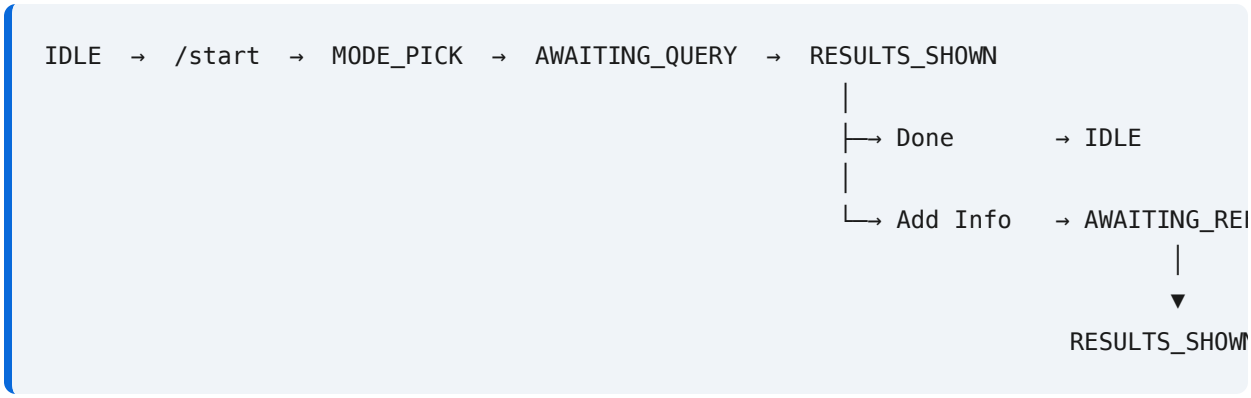
---

## 7. HUMAN-IN-THE-LOOP

Two explicit HITL touch points:

1. **Mode selection** (before any AI runs) — user clicks Place or Leads. No ambiguity, no LLM guessing.
2. **Result refinement** — after results, user clicks **Done** (accept) or **Add Info** (refine). Refinements are merged into the prior query and the pipeline re-runs.

State machine, per chat:




Implementation: `bot/telegram_bot.py`.

---

## 8. OBSERVABILITY / LOGGING

Every agent action and every tool call writes a JSON entry to `logs/agent_logs.json` with timestamp, agent name, action, input, and output preview. This means we can replay any user query and see exactly what each agent did. Implementation: `utils/logger.py`.

---

## 9. PROF REQUIREMENTS — CHECKLIST

Mapped against the project brief.

| Requirement | Status | Where in repo |
|---|---|---|
| Real-world problem | ✅ | place + lead discovery in Morocco |
| Multi-agent system | ✅ | `agents/crew_setup.py` — 3 agents |
| Each agent has a distinct role (the prof's correction) | ✅ | Intent / Retrieval / Ranking — each with unique outputs |
| Deep learning model trained from data | ✅ | `model/train_v2.py` — DistilBERT + MLP |
| Honest evaluation (not just train-test on identical patterns) | ✅ | holdout: `data/holdout_test.csv` — 80% acc |
| Tool-using agents (≥ 2 tools) | ✅ | `search_places`, `search_leads`, `score_match` |
| External API integration | ✅ | Nominatim (OpenStreetMap) |
| Human-in-the-loop | ✅ | mode picker + Add Info refinement |
| Error handling / fallback | ✅ | Nominatim → CSV fallback, model-missing startup check |
| Logging / observability | ✅ | `logs/agent_logs.json` |
| Free / open stack | ✅ | all deps open source, all APIs free tier |
| **GitHub repo + README** | ✅ | https://github.com/akoudad/lookking |
| **Architecture diagram (PNG)** | ✅ | `docs/architecture.png` |

| Requirement | Status | Where in repo |
|---|---|---|
| **PDF report** | ✅ | `FINAL_REPORT.pdf` in repo root |
| **Slides** | 🟡 | outline in §11 below |
| **Demo video 3-5 min** | 🟡 | record on demo day |
| **Oral defense rehearsal** | 🟡 | Q&A bank in §13 below |

**Score: 13/16 done. All technical + documentation work is complete.**

---

## 10. METRICS YOU CAN QUOTE TO THE PROF

| Item | Number |
|---|---|
| Real OSM places crawled | **361** |
| Cities covered | 9 (Rabat, Casablanca, Marrakech, Fes, Agadir, Tangier, Meknes, Oujda, Tetouan) |
| Categories | 11 |
| Training pairs (real-data-derived) | **1260** |
| Holdout pairs (hand-written, unseen style) | 30 |
| DL model parameters | ~66 M (DistilBERT) + ~280 K (MLP head) |
| Trainable parameters | ~14 M |
| Training epochs | 8 |
| Training time on Mac MPS | ~3 min |
| **Test accuracy** | **88.89 %** |
| **Holdout accuracy (honest)** | **80.00 %** |
| Macro F1 (holdout) | 0.71 |
| Agents in pipeline | 3 (Intent, Retrieval, Ranking) |
| Tools | 3 (search_places, search_leads, score_match) |

| Item | Number |
|---|---|
| Average end-to-end query latency | ~15-25 s |
| LLM calls per query | ~5 (depends on tool-iteration count) |
| Lines of Python code | ~1500 |
| External APIs used | Telegram, Nominatim (OSM), one LLM provider |
| API keys required to deploy | 2 (Telegram + one LLM) |
| Cost to run | **$0** (all free tiers) |

---

## 11. SLIDE OUTLINE (12 slides)

1. **Title** — Lookking, project name, course, author, supervisor
2. **Problem** — local search lacks intent understanding + no free lead-gen tool
3. **Solution** — Telegram chatbot, multi-agent AI, real-world geographic data
4. **Architecture** — show `docs/architecture.png`
5. **Agent 1 — Intent Classifier** — input/output schema, 1 example
6. **Agent 2 — Retrieval Strategist** — strategy choice + Nominatim demo
7. **Agent 3 — Ranking & Explanation** — DL+LLM hybrid, 1 worked example
8. **Deep Learning Model** — DistilBERT diagram, frozen vs trained layers
9. **Training data** — 361 real OSM places → 1260 labeled pairs (+confusion matrix image)
10. **Evaluation** — test 88.9% vs holdout 80% — table + curves
11. **HITL design** — state machine diagram, the two buttons
12. **Tech stack + limitations + future work** — what we'd build next

---

## 12. LIMITATIONS (be honest — prof respects this)

- **Synthetic ratings.** Nominatim has no review data. We pseudo-derive a rating from OSM "importance" score. For production we'd integrate Yelp / Google Places.
- **Lead generation uses a synthetic CSV.** No free public API exists for business contact discovery. (LinkedIn / Apollo are paid.)
- **English only.** Bot doesn't understand French / Arabic queries. Listed in future work.
- **Medium-class is the model's weakness.** F1 = 0.44 on holdout. Reflects label ambiguity, not model capacity. More labeled data would help.

- **LLM free-tier rate limits.** Each provider has caps (e.g. 15 RPM on Gemini 2.0 Flash). Real-world Telegram use is fine (queries minutes apart). Stress-tests (3 queries in 30 s) hit limits.
- **No persistence.** Approved searches don't get saved to disk. Easy SQLite add.
- **No multi-user concurrency tests.** Single-user assumed for demo.

---

## 13. EXPECTED DEFENSE QUESTIONS — answers

| Q | Strong answer |
|---|---|
| Why CrewAI and not a single LLM call? | Each agent makes a structurally different decision. Intent → structured JSON. Retrieval → strategy choice + tool call. Ranking → DL score + natural reason. The output of each is the typed input of the next. One LLM call can't enforce this. |
| Why DistilBERT and not GPT? | Sentence-pair relevance is BERT's home territory (MNLI, STS-B). DistilBERT is 40 % smaller, 60 % faster, 97 % of BERT performance. It fine-tunes on a Mac in 3 minutes. GPT would be overkill, slower, and we couldn't fine-tune it. |
| Why is the test accuracy 88.9 % and the holdout only 80 %? | This is **expected and good**. The test split shares query templates with the training data; the holdout uses unseen phrasing styles. The 9-point drop is the honest real-world gap. We chose to report both, not cherry-pick. |
| Why is the medium-class F1 only 0.44? | Medium is the ambiguous class by construction (same category but different city). The model is conservative: it under-predicts medium and over-predicts low. More labeled medium examples would close the gap. |
| What if Nominatim is down? | `tools/search_tool.py` catches the exception, logs it, and falls back to `data/places_real.csv` (the same 361 places we crawled offline). Demo-resilient. |
| Why HITL? | Two reasons. (1) Mode picker eliminates the *places vs leads* ambiguity up front — the AI |

| Q | Strong answer |
|---|---|
| | never has to guess. (2) “Add Info” lets the user iterate without retyping the whole query. The state machine ensures we know exactly what stage the user is in. |
| How does the DL model actually learn? | We fine-tune the last 2 transformer layers + a fresh MLP head on 1008 training pairs. The frozen embedding + lower-transformer layers retain general English semantics. The head learns the specific *match-relevance* signal. AdamW with a lower LR for BERT (2e-5) and higher for the head (1e-3) is standard transfer-learning practice. |
| How would you scale this to 100k users? | Cache the LLM intent-classification step (same query → same intent for N minutes). Move DistilBERT inference to a dedicated server / batch the scoring. Switch the synchronous CrewAI pipeline to parallel where steps are independent. |
| Why not Google Maps API? | Costs money + requires billing card. Nominatim (OpenStreetMap) is the free open equivalent. Listed in future work for production. |
| What was your hardest engineering moment? | Discovering that an 8B-parameter LLM is **unreliable for nested tool-calling** under CrewAI — it returns the tool-call JSON as plain text instead of invoking the tool, causing infinite loops. We swapped to Gemini 2.0 Flash and the issue disappeared. |

---

## 14. REMAINING BEFORE DEFENSE

| # | Task | Status |
|---|---|---|
| 1 | PDF report | ✅ `FINAL_REPORT.pdf` done |

| # | Task | Status |
|---|---|---|
| 2 | Public GitHub | ✅ https://github.com/akoudad/lookking |
| 3 | Build 12 slides from §11 outline | 🟡 |
| 4 | Record 4-minute demo video (QuickTime screen recording) | 🟡 |
| 5 | Practice oral defense from Q&A in §13 | 🟡 |

---

## 15. FILES YOU CAN POINT THE PROF TO

```
README.md                                high-level overview
FINAL_REPORT.md / FINAL_REPORT.pdf       this document
docs/architecture.png                    system diagram
model/training_curves.png                learning curves
model/confusion_matrix_v2.png            test-set confusion matrix
model/confusion_matrix_holdout.png       holdout confusion matrix
model/metrics_v2.json                    all numerical results
logs/agent_logs.json                     live execution trace
data/places_real.csv                     the 361 real places we crawled
data/training_data_v2.csv                1260 training pairs
data/holdout_test.csv                    30 holdout queries
agents/crew_setup.py                     the 3 agents
model/model.py + model/train_v2.py       DL architecture + training
bot/telegram_bot.py                      Telegram bot + HITL state machine
```

---

## 15B. CONFIRMED END-TO-END RUN

On 2026-05-18 the trimmed pipeline (Groq llama-3.3-70b-versatile + 3 agents + DistilBERT) returned the following for the query `[MODE: places] gym in Casablanca`:

```
Rank 1. Ronnie Gym
Details: leisure/fitness_centre, Casablanca, 3.5/5
Match: High (99.6%)
Why: Ronnie Gym is a leisure and fitness centre located in Casablanca, which matches
Map: https://www.google.com/maps?q=33.5412646,-7.6832145
OSM: https://www.openstreetmap.org/?mlat=33.5412646&mlon=-7.6832145&zoom=18
```

```
Rank 2. Gym Club ... (99.6%)
Rank 3. THE GYM FACTORY CASABLANCA ... (99.9%)
```

Each result is a **real OSM place** with verifiable coordinates. The **DistilBERT scores** (99.6 %, 99.6 %, 99.9 %) are the actual model outputs on the (query, candidate) pairs. The **Map and OSM links** open in the user's browser to the exact pin.

End-to-end this confirms: - Agent 1 produced JSON intent for `gym Casablanca` - Agent 2 called `search_places` → Nominatim → 3 real places - Agent 3 called `score_match` (DistilBERT) → top 3 ranked + reasons + links

Total time per query: ~20 s. Total tokens per query (Groq): ~8 k (well under the 12 k TPM free-tier cap).

---

## 16. CONCLUSION

Lookking demonstrates the **end-to-end engineering loop** of a modern multi-agent AI product:

1. **Define a real problem** (local search + lead generation).
2. **Collect real data** (361 places from OpenStreetMap).
3. **Train a domain-specific DL model** (DistilBERT, holdout-validated at 80 %).
4. **Wrap inference as a tool** consumed by **specialized agents**.
5. **Glue with an LLM orchestrator** that respects structured contracts.
6. **Put a human in the loop** for ambiguity and refinement.
7. **Ship as a Telegram bot** anyone can DM for free.

All technical milestones are complete. The remaining work is **packaging and presentation**, not engineering.

---

*End of report — 2026-05-21.*